Index

Symbols

- 2PC (two-phase commit algorithms), Distributed Transactions—Two-Phase Commits-Distributed Transactions—Two-Phase Commits

A

- A/B tests, A/B testing
- ACID (atomicity, consistency, isolation, and durability), ACID Transactions
- Active Directory, Common Single Sign-On Implementations
- adaptability, Sustained Adaptability, Chaos Engineering, Summary, Summary
- aggregate, Aggregate-Aggregate, Mapping Aggregates and Bounded Contexts to Microservices, Glossary
- alarms, versus alerts, Alert fatigue
- alert fatigue, Alert fatigue
- alerting, Alerting-Toward better alerting
- Amazon Web Services (AWS)
  - API gateways, Service Meshes and API Gateways
  - autonomous product-oriented teams, Ensuring Consistency
  - autoscaling in, Desired State Management, Autoscaling-Autoscaling
  - availability zones, Spreading Your Risk
  - AWS Lambda, Windows containers
  - AWS Secrets Manager, Secrets
  - Beanstalk, Deployment Options
  - client libraries, Client libraries
  - CloudWatch, Implementations
  - containers and, Isolated, differently
  - credentials and security, User credentials, Rotation, Backups
  - defined, Glossary
  - desired state management and, Desired State Management
  - dynamic service registries, Rolling your own
  - Elasticsearch and, Implementations
  - embracing failure, Production Experiments

    - FaaS services, Deployment Options, Challenges
    - in-the-cloud development, Developer Experience
    - isolated execution, Isolated Execution
    - limitations of, Limitations
    - managed VMs on, Good for microservices?
    - market share of, Multitenancy and Federation
    - message brokers, Choices
    - on-demand provisioning systems, Scaling
    - platform-specific tools, Infrastructure as Code (IAC)
    - Relational Database Service (RDS), Database deployment and scaling
    - selecting, Should You Use It?
    - SLA availability, Multiple Instances, Service-level agreement, Redundancy
    - type 2 virtualization, Cost of virtualization
    - vertical scaling, Implementation
- anomaly detection, The Expert in the Machine
- Ansible, Which Deployment Option Is Right for You?
- antifragility, And the Real World
- Apache Flink, Streaming
- Apache Kafka, Streaming
- API gateways, Keep Your APIs Technology Agnostic, Service Meshes and API Gateways-What to avoid, Glossary
- application containers, Application Containers
- application security (see also security)
    - ability to rebuild, Rebuild
    - backups, Backups
    - credentials, Credentials-Limiting scope
    - patching, Patching-Patching
- application state, hypermedia as the engine of, Hypermedia as the engine of application state
- architects, What's in a Name?
    - (see also evolutionary architects)
- architectural principles
    - defining standards for microservices, The Required Standard-Architectural Safety
    - guiding evolutionary architecture, Guiding an Evolutionary Architecture
    - heterogeneous architecture, Technology Heterogeneity
    - layering inside versus outside, Organizational
    - principles and practices, A Principled Approach-A Real-World Example
    - self-contained systems (SCSs), When to Use It
    - software architecture defined, What Is Software Architecture?

    - three-tiered architecture, Alignment of Architecture and Organization-Alignment of Architecture and Organization
    - vertical architecture, Alignment of Architecture and Organization
- architectural safety, Architectural Safety
- artifact creation, Artifact Creation
- async/await construct, Disadvantages
- AsyncAPI, Explicit Interface
- asynchronous nonblocking calls, Pattern: Asynchronous Nonblocking-Where to Use It
- atomic commits, versus atomic deployment, Pattern: Monorepo
- atomicity, consistency, isolation, and durability (ACID), ACID Transactions
- authentication and authorization
    - centralized, upstream authorization, Centralized, Upstream Authorization
    - common single sign-on (SSO), Common Single Sign-On Implementations
    - confused deputy problem, The Confused Deputy Problem-The Confused Deputy Problem
    - decentralized authorization, Decentralizing Authorization
    - defined, Authentication and Authorization, Glossary
    - fine-grained authorization, Fine-Grained Authorization
    - human authentication, Human Authentication
    - JSON Web Token (JWT), JSON Web Tokens-Challenges
    - mutual authentication, Client identity
    - service-to-service authentication, Service-to-Service Authentication
    - single sign-on gateways, Single Sign-On Gateway-Single Sign-On Gateway
- automation
    - in anomaly detection, The Expert in the Machine
    - as essential to security, Automation
    - during deployment, Focus on Automation
- autonomy, On Autonomy, Summary
- autoscaling, Autoscaling
- availability, How Much Is Too Much?, Sacrificing Availability
- Azure
    - Application Insights, Implementations
    - Azure Functions, Deployment Options, Limitations
    - Azure Web Apps, Should You Use It?
    - Azure's Key Vault, Secrets
    - benefits of, Should You Use It?
    - defined, Glossary
    - desired state management, Desired State Management
    - event grid product, Explicit Interface

  - limitations of, Limitations-Limitations
  - managed VMs on, Good for microservices?
  - market share of, Multitenancy and Federation
  - openapi-diff tool, Catch Accidental Breaking Changes Early
  - public cloud services offered by, Public Cloud and Serverless
  - SLA availability, Multiple Instances

B

- backend for frontend (BFF) pattern, Pattern: Backend for Frontend (BFF)-When to Use
- Backstage tool, The Self-Describing System
- backups, Backups, Encrypt backups
- backward compatibility, Make Backward Compatibility Easy (see also breaking changes)
- backward recovery, versus forward, Saga Failure Modes
- Biz Ops tool, The Self-Describing System
- blameless post-mortems, Blame
- blue-green deployment, Parallel Run
- bounded context
  - alternatives to business domain boundaries, Alternatives to Business Domain Boundaries-Organizational
  - defined, Bounded Context, Glossary
  - defining system boundaries, Defining System Boundaries-Defining System Boundaries
  - hidden models, Hidden models
  - mapping to microservices, Mapping Aggregates and Bounded Contexts to Microservices
  - shared models, Shared models
- BPM (business process modeling) tools, Orchestrated sagas
- Brakeman, Build Security into the Delivery Process
- branching, Branching Models
- breaking changes
  - avoiding, Avoiding Breaking Changes-Catch Accidental Breaking Changes Early
  - managing, Managing Breaking Changes-Extreme Measures
- brittle tests, Flaky and Brittle Tests
- brittleness, Brittleness
- build pipelines, Build Pipelines and Continuous Delivery-Build Pipelines and Continuous Delivery
- bulkheads, Stability Patterns, Bulkheads, Glossary

- business functionality, aligning architecture with, Alignment of Architecture and Organization-Alignment of Architecture and Organization
- business process modeling (BPM) tools, Orchestrated sagas
- business-facing tests, Types of Tests

C

- caching
  - basics of, Caching
  - cache poisoning, Cache Poisoning: A Cautionary Tale
  - invalidation, Invalidation-Write-behind
  - performance benefits, For Performance
  - robustness benefits, For Robustness
  - scaling benefits, For Scale
  - trade-offs between freshness and optimization, The Golden Rule of Caching-Freshness Versus Optimization
  - where to cache, Where to Cache-Request cache
- canary releases, Canary Release, Canary release
- CAP theorem
  - availability, Sacrificing Availability
  - combined approach, It's Not All or Nothing
  - components of, CAP Theorem-CAP Theorem
  - consistency, Sacrificing Consistency
  - partition tolerance, Sacrificing Partition Tolerance?
  - real-world application of, And the Real World
  - selecting your approach, AP or CP?
- cardinality, low versus high, Low versus high cardinality
- CDCs (consumer-driven contracts), Catch Accidental Breaking Changes Early, Contract Tests and Consumer-Driven Contracts (CDCs)-It's about conversations
- central aggregating gateway pattern, Pattern: Central Aggregating Gateway-When to Use It
- CFR (cross-functional requirements), Cross-Functional Testing, How Much Is Too Much?, Glossary
- change reviews, Change Reviews
- chaos engineering, Chaos engineering, Chaos Engineering-From Robustness to Beyond
- Chaos Monkey, Production Experiments
- Chaos Toolkit, From Robustness to Beyond
- Chef, Which Deployment Option Is Right for You?

- choreographed sagas, Implementing Sagas, Choreographed sagas-Should I use choreography or orchestration (or a mix)?, Workflow
- choreography, Glossary
- circuit breakers, Stability Patterns, Circuit Breakers-Circuit Breakers, Glossary
- client identity, Client identity
- client libraries, Client libraries
- client-side caching, Client-side-Client-side
- closed source development, Branching Models
- Cloud Native Computing Foundation (CNCF), The Cloud Native Computing Federation
- CloudEvents, Explicit Interface
- COBIT (Control Objectives for Information Technologies), Governance and the Paved Road
- code branching, Branching Models
- code organization
  - one repository for multiple microservices, Pattern: Monorepo-Where to use this pattern
  - one repository per microservice, Pattern: One Repository per Microservice (aka Multirepo)-Where to use this pattern
  - single, giant repository, One Giant Repo, One Giant Build-One Giant Repo, One Giant Build
- code reuse, DRY and the Perils of Code Reuse in a Microservice World-Client libraries, Reusing code across repositories
- code reviews, Change Reviews-Synchronous versus asynchronous code reviews
- cohesion
  - coupling and, The Interplay of Coupling and Cohesion
  - defined, Cohesion, Glossary
- collaboration, Summary
- collective ownership, Collective Ownership-Collective Ownership, Glossary
- Command Query Responsibility Segregation (CQRS) pattern, Start Small
- commands, versus requests, Pattern: Request-Response Communication
- comments and questions, How to Contact Us
- common coupling, Common Coupling-Common Coupling
- common single sign-on (SSO), Common Single Sign-On Implementations
- communication styles
  - asynchronous nonblocking, Pattern: Asynchronous Nonblocking-Where to Use It
  - commands versus requests, Pattern: Request-Response Communication
  - communicating through common data, Pattern: Communication Through Common Data-Where to Use It

  - complexity introduced by, Proceed with Caution
  - event-driven, Pattern: Event-Driven Communication-Where to Use It
  - in-process versus inter-process calls, From In-Process to Inter-Process-Error Handling
  - mixing styles, Mix and Match
  - overview of, Styles of Microservice Communication, Communication Styles
  - parallel versus sequential calls, Implementation: Synchronous Versus Asynchronous
  - request-response, Pattern: Request-Response Communication-Where to Use It
  - synchronous blocking, Pattern: Synchronous Blocking-Where to Use It
  - technology supporting, Technology for Inter-Process Communication: So Many Choices
- communication, defined, Architecture in a Stream-Aligned Organization
- community of practice (CoP) groups, Communities of Practice
- compatibility, assuring backward, Make Backward Compatibility Easy (see also breaking changes)
- compensating transactions, Saga rollbacks
- composability, Composability
- conditional GETs, Conditional GETs
- Confluent Schema Registry, Catch Accidental Breaking Changes Early
- confused deputy problem, The Confused Deputy Problem-The Confused Deputy Problem
- consistency, Sacrificing Consistency, Ensuring Consistency
- Consul, Consul
- consumer-driven contracts (CDCs), Catch Accidental Breaking Changes Early, Contract Tests and Consumer-Driven Contracts (CDCs)-The Final Word
- consumer-first approach, The Social Contract
- container orchestration, The Case for Container Orchestration (see also Kubernetes)
- container-based virtualization, Cost of virtualization
- containers (see also application containers)
  - basics of, Isolated, differently-Isolated, differently
  - defined, Glossary
  - Docker, Docker
  - drawbacks of, Not perfect
  - fitness for microservices, Fitness for microservices
  - role in microservices, Containers and Kubernetes, Containers
  - security concerns, Patching
  - Windows and, Windows containers
- content coupling, Content Coupling-Content Coupling
- context, monitoring tools providing, Provide Context

- continuous delivery (CD), Build Pipelines and Continuous Delivery-Build Pipelines and Continuous Delivery, Glossary
- continuous deployment, Build Pipelines and Continuous Delivery, Glossary
- continuous integration (CI), A Brief Introduction to Continuous Integration-Branching Models, Glossary
- contract breakages
  - avoiding breaking changes, Avoiding Breaking Changes-Catch Accidental Breaking Changes Early
  - managing, Managing Breaking Changes-Extreme Measures
  - structural versus semantic, Structural Versus Semantic Contract Breakages
- contract tests, Contract Tests and Consumer-Driven Contracts (CDCs)
- Control Objectives for Information Technologies (COBIT), Governance and the Paved Road
- Conway's law, Alignment of Architecture and Organization, Conway's Law-Netflix and Amazon, Conway's Law in Reverse, Glossary
- CoP (community of practice) groups, Communities of Practice
- core committers, Internal Open Source-Role of the Core Committers
- costs, Cost
- COTS (customizable off the shelf software), Advantages, Glossary
- coupling
  - cohesion and, The Interplay of Coupling and Cohesion
  - common coupling, Common Coupling-Common Coupling
  - content coupling, Content Coupling-Content Coupling
  - defined, Glossary
  - domain coupling, Domain Coupling
  - loose versus tight, Coupling
  - pass-through coupling, Pass-Through Coupling-Pass-Through Coupling
  - pathological coupling, Content Coupling
  - technology coupling, Technology coupling
  - temporal coupling, Domain Coupling
  - types of, Types of Coupling
- CQRS (Command Query Responsibility Segregation) pattern, Start Small
- credentials
  - accidentally including keys in source code, Revocation
  - challenges of microservices, Credentials
  - limiting scope of, Limiting scope-Limiting scope
  - revoking, Revocation
  - rotating frequently, Rotation
  - secrets, Secrets-Secrets
  - user credentials, User credentials

- cross-functional requirements (CFR), Cross-Functional Testing, How Much Is Too Much?, Glossary
- cross-functional testing, Cross-Functional Testing-Robustness Tests
- custom resource definitions (CRDs), Helm, Operators, and CRDs, Oh My!
- customizable off the shelf software (COTS), Advantages, Glossary
- cybersecurity, five functions of, The Five Functions of Cybersecurity-Recover
  - (see also security)

D

- data
  - communicating through common data, Pattern: Communication Through Common Data-Where to Use It
  - decomposition concerns, Data Decomposition Concerns-Reporting Database
  - durability of, How Much Is Too Much?
  - high-cardinality data, Low versus high cardinality
  - influence on decomposition, Data
  - maintaining consistency of, Data Consistency
  - securing, Securing Data-Encrypt backups
- data partitioning, Data Partitioning-Limitations, Glossary
- database transactions
  - ACID transactions, ACID Transactions
  - defined, Database Transactions
  - lacking atomicity, Still ACID, but Lacking Atomicity?-Still ACID, but Lacking Atomicity?
- databases
  - deployment concerns, The Database-Environments
  - hiding, What Are Microservices?
  - integrity concerns, Data Integrity
  - performance concerns, Performance-Performance
  - reporting databases, Reporting Database
  - shared, Owning Their Own State
  - tooling concerns, Tooling
  - transaction concerns, Transactions
- Debezium, Streaming
- decentralization, Loosely Coupled Organizations
- decentralized authorization, Decentralizing Authorization
- decomposition (see also under user interfaces (UIs))
  - combined approach to, The Monolith Is Rarely the Enemy
  - dangers of premature, The Dangers of Premature Decomposition

  - data concerns, Data Decomposition Concerns-Reporting Database
  - goal setting, Have a Goal, Strategic Goals
  - incremental migrations, Incremental Migration
  - influence of data on, Data
  - layers of, Decomposition by Layer-Data First
  - patterns for, Useful Decompositional Patterns
  - selecting a starting point, What to Split First?-What to Split First?
  - volatility-based, Volatility
- defense in depth, Defense in Depth
- delivery bottlenecks, Delivery Bottlenecks, Pluggable, Modular Microservices
- delivery contention, Monoliths and Delivery Contention
- deployment
  - application containers, Application Containers
  - blue-green deployment, Parallel Run
  - container orchestration and Kubernetes, Kubernetes and Container Orchestration-Should You Use It?
  - containers, Containers-Fitness for microservices
  - continuous deployment, Build Pipelines and Continuous Delivery, Glossary
  - database concerns, The Database
  - ease of with microservices, Ease of Deployment
  - environments, Environments-Environments
  - Function as a Service (FaaS), Function as a Service (FaaS)-The way forward
  - in Kubernetes, A Simplified View of Kubernetes Concepts
  - lockstep deployment, Lockstep Deployment
  - moving from logical to physical, From Logical to Physical
  - multiple instances of each service, Multiple Instances
  - options for, Deployment Options
  - overview of, Deployment
  - physical machines, Physical Machines
  - Platform as a Service (PaaS), Platform as a Service (PaaS)
  - principles of, Principles of Microservice Deployment-GitOps, Summary
  - progressive delivery, Progressive Delivery-Parallel Run
  - role of Puppet, Chef, and other tools, Which Deployment Option Is Right for You?
  - selecting the right deployment platform, Which Deployment Option Is Right for You?, Summary
  - virtual machines (VMs), Virtual Machines-Good for microservices?
  - zero-downtime deployment, Zero-Downtime Deployment
- desired state management, Desired State Management-GitOps
- detective controls, Defense in Depth, Glossary

- development
  - artifact creation, Artifact Creation
  - build pipelines, Build Pipelines and Continuous Delivery-Build Pipelines and Continuous Delivery
  - building security into software delivery, Build Security into the Delivery Process
  - challenges of, Developer Experience
  - code organization, Mapping Source Code and Builds to Microservices-Which Approach Would I Use?
  - continuous delivery (CD), Build Pipelines and Continuous Delivery-Build Pipelines and Continuous Delivery, Glossary
  - continuous integration (CI), A Brief Introduction to Continuous Integration-Branching Models, Glossary
  - tooling, Tooling
  - trade-offs and environments, Trade-Offs and Environments
- directory services, Common Single Sign-On Implementations
- distributed monoliths, The Distributed Monolith
- distributed tracing, Log Aggregation and Distributed Tracing, Distributed Tracing-Implementing distributing tracing
- distributed transactions
  - avoiding, Distributed Transactions—Just Say No
  - versus sagas, Sagas Versus Distributed Transactions
  - two-phase commits, Distributed Transactions—Two-Phase Commits-Distributed Transactions—Two-Phase Commits
- DNS (Domain Name System), Domain Name System (DNS)
- Docker, Docker, Glossary
- Dockerfiles, Which Deployment Option Is Right for You?
- documentation, of services, Documenting Services-The Self-Describing System
- domain coupling, Domain Coupling, Glossary
- domain events, The process
- Domain Name System (DNS), Domain Name System (DNS)
- domain-driven design (DDD), Modeled Around a Business Domain, Just Enough Domain-Driven Design-The Case for Domain-Driven Design for Microservices, What Are Microservices?, Glossary
- DRY (don't repeat yourself), DRY and the Perils of Code Reuse in a Microservice World
- dynamic service registries, Dynamic Service Registries-Don't Forget the Humans!

E

- ease of deployment, Ease of Deployment

- EEMUA (Engineering Equipment and Materials Users Association), Toward better alerting
- Elasticsearch, Implementations
- empathy, Summary
- enabling teams, Sharing Specialists, Enabling Teams-The paved road, Architecture in a Stream-Aligned Organization
- end-to-end tests
  - alternatives to, Should You Avoid End-to-End Tests?-The Final Word
  - implementing, Implementing (Those Tricky) End-to-End Tests-Lack of Independent Testability
  - scope of, End-to-End Tests
- Engineering Equipment and Materials Users Association (EEMUA), Toward better alerting
- engineers, What's in a Name?
  - (see also evolutionary architects)
- ensemble programming, Ensemble programming
- environments
  - during deployment, Environments-Environments
  - during development, Trade-Offs and Environments
- error budgets, Error budgets, Glossary
- error handling, Error Handling, Exception Handling
- etcd, etcd and Kubernetes
- event sourcing, Start Small
- event storming, Event Storming-The process
- event-driven communication, Pattern: Event-Driven Communication-Where to Use It, Communication Styles
- events
  - components of, What's in an Event?-Fully detailed events
  - defined, Glossary
  - versus messages, Pattern: Event-Driven Communication
- evolutionary architects
  - architecture in stream-aligned organizations, Architecture in a Stream-Aligned Organization
  - building teams, Building a Team
  - concept of, Architecture
  - core responsibilities of, Summary
  - defining standards for microservices, The Required Standard-Architectural Safety
  - defining system boundaries, Defining System Boundaries-Defining System Boundaries

  - exception handling, Exception Handling
  - governance and the paved road, Governance and the Paved Road-The Paved Road at Scale
  - guiding evolutionary architecture, Guiding an Evolutionary Architecture
  - habitability and, Habitability
  - making change possible, Making Change Possible
  - principles and practices, A Principled Approach-A Real-World Example
  - role of, What's in a Name?-What's in a Name?
  - social construct of, A Social Construct
  - software architecture defined, What Is Software Architecture?
  - technical debt and, Technical Debt
  - vision for, An Evolutionary Vision for the Architect
- explicit schemas, Make Your Interface Explicit, Should You Use Schemas?, Explicit Interface, Explicit Schemas
- exploratory testing, Types of Tests
- extensibility, graceful, Graceful Extensibility

F

- feature branching, Branching Models, Glossary
- feature toggles, Feature Toggle, Feature Toggles
- federation (Kubernetes), Multitenancy and Federation-Multitenancy and Federation
- fine-grained authorization, Fine-Grained Authorization
- Firecracker, Windows containers
- fitness functions, Guiding an Evolutionary Architecture
- flaky tests, Flaky and Brittle Tests
- flexibility, Flexibility
- Fluentd, Implementations
- focus on automation, Focus on Automation
- forward recovery, versus backward, Saga Failure Modes
- fragility, And the Real World
- frameworks, Tailored Microservice Template
- frontend teams, Drivers for Dedicated Frontend Teams
- full stack teams, Toward Stream-Aligned Teams
- full-fat virtualization, Isolated, differently
- Function as a Service (FaaS), Public Cloud and Serverless, Function as a Service (FaaS)-The way forward, Glossary
- functional decomposition, Functional Decomposition-Limitations

G

- game day exercises, Game Days
- GitFlow development model, Branching Models
- GitOps, GitOps
- global consistency, Balancing Models
- golden rule of caching, The Golden Rule of Caching
- governance, Governance and the Paved Road-The Paved Road at Scale, Summary, Glossary
- graceful extensibility, Graceful Extensibility, Glossary
- Graphite, Implementations
- GraphQL, GraphQL-Where to use it, GraphQL-GraphQL, Glossary
- guaranteed delivery, Guaranteed delivery

H

- habitability, Habitability
- hash-based message authentication code (HMAC), Manipulation of data
- HATEOAS (hypermedia as the engine of application state), Hypermedia as the engine of application state
- Helm, Helm, Operators, and CRDs, Oh My!
- Heroku's Twelve Factors, Principles
- heterogeneous architectures, Technology Heterogeneity
- Hexagonal Architecture pattern, Microservices at a Glance
- hidden models, Hidden models
- high-cardinality data, Low versus high cardinality
- HMAC (hash-based message authentication code), Manipulation of data
- Honeycomb, Log Aggregation and Distributed Tracing, Implementations
- horizontal architecture, Organizational
- horizontal duplication, Horizontal Duplication-Limitations, Glossary
- HTTP (Hyper Text Transfer Protocol)
  - REST and, REST and HTTP
- human authentication, Human Authentication
- humane registry, The Self-Describing System
- Hyper-V containers, Windows containers
- hypermedia as the engine of application state (HATEOAS), Hypermedia as the engine of application state
- hypervisors, Cost of virtualization

I

- idempotency, Idempotency, Glossary
- identity providers, Common Single Sign-On Implementations
- implementation
  - API gateways, Service Meshes and API Gateways-What to avoid
  - code reuse, DRY and the Perils of Code Reuse in a Microservice World-Client libraries
  - documenting services, Documenting Services-The Self-Describing System
  - goal setting, Looking for the Ideal Technology-Hide Internal Implementation Detail, Strategic Goals
  - handling change between microservices, Handling Change Between Microservices
  - schemas, Schemas-Should You Use Schemas?
  - serialization formats, Serialization Formats
  - service discovery, Service Discovery-Don't Forget the Humans!
  - service meshes, Service Meshes and API Gateways-Service Meshes and API Gateways, Service Meshes-What About Other Protocols?
  - technology choices, Technology Choices-Kafka
- implicit trust, Implicit Trust
- in-process versus inter-process calls, From In-Process to Inter-Process-Error Handling
- in-production testing, Types of In-Production Testing, Testing in Production-Chaos engineering
- independent deployability, Independent Deployability, Owning Their Own State, Lack of Independent Testability, What Are Microservices?, Glossary
- information hiding, Microservices at a Glance, Owning Their Own State, Information Hiding, What Are Microservices?, Glossary
- infrastructure as code (IAC), Infrastructure as Code (IAC), Glossary
- integration tests, End-to-End Tests
- interfaces, exposing explicit, Make Your Interface Explicit, Should You Use Schemas?, Explicit Interface, Explicit Schemas
- internal open source structure, Internal Open Source-Tooling
- invalidation
  - conditional GETs, Conditional GETs
  - notification-based, Notification-based
  - time to live (TTL), Time to live (TTL)
  - write-behind caches, Write-behind
  - write-through caches, Write-through
- isolated execution, Isolated Execution-Isolated Execution

- isolation, Isolation
- Istio, How they work, And Knative
- IT architects, An Evolutionary Vision for the Architect
  - (see also evolutionary architects)

J

- Jaeger, Log Aggregation and Distributed Tracing
- JSON Web Token (JWT), JSON Web Tokens-Challenges, Glossary
- json-schema-diff-validator, Catch Accidental Breaking Changes Early

K

- Kafka, Kafka
- key performance indicators (KPIs), Toward Stream-Aligned Teams
- keys
  - accidentally including in source code, Revocation
  - scanning for, Revocation
  - secure storage of, It's all about the keys
- Kibana, Implementations
- Kinesis, Choices
- Knative, And Knative
- KSQLDB, Streaming
- Kubernetes
  - background of, Multitenancy and Federation
  - basics of, A Simplified View of Kubernetes Concepts-A Simplified View of Kubernetes Concepts
  - benefits and drawbacks of, Should You Use It?
  - Cloud Native Computing Foundation (CNCF), The Cloud Native Computing Federation
  - configuration management in, etcd and Kubernetes
  - defined, Glossary
  - future development, The Future
  - Knative and, And Knative
  - managing third-party applications, Helm, Operators, and CRDs, Oh My!
  - multitenancy and federation, Multitenancy and Federation-Multitenancy and Federation
  - platforms and portability, Platforms and Portability
  - role in microservices, Containers and Kubernetes
  - secrets management with, Secrets

L

- latency, Latency, How Much Is Too Much?
- libraries
  - client libraries, Client libraries
  - defined, Glossary
  - external contributions through, External contribution through libraries
  - sharing code via, Sharing Code via Libraries
- Lightstep, Log Aggregation and Distributed Tracing, Implementations
- Lightweight Directory Access Protocol (LDAP), Common Single Sign-On Implementations
- load shedding, Bulkheads
- local calls, versus remote calls, Local calls are not like remote calls
- local optimization, Balancing Models
- lockstep deployment, Lockstep Deployment, Glossary
- log aggregation
  - basics of, Log Aggregation-Log Aggregation
  - common format for, Common format
  - correlating log lines, Correlating log lines-Correlating log lines
  - implementations, Implementations
  - role in microservices, Log Aggregation and Distributed Tracing
  - shortcomings of, Shortcomings
  - timing, Timing
- long lived transactions (LLTs), Sagas
- loose coupling, Independent Deployability, Coupling
- loosely coupled organizations, Loosely Coupled Organizations, Loosely and tightly coupled organizations

M

- managed brokers, Choices
- manual exploratory testing, Types of Tests
- mean time between failures (MTBF), Mean Time to Repair over Mean Time Between Failures?
- mean time to repair (MTTR), Mean Time to Repair over Mean Time Between Failures?
- MELT (metrics, event, logs, and traces), The Pillars of Observability? Not So Fast
- message brokers, Message Brokers-Kafka, Glossary
- messages, Glossary
- metrics aggregation, Metrics Aggregation-Implementations

- micro frontend pattern, Pattern: Micro Frontends-When to Use It
- microservices (see also modeling microservices)
  - approach to learning, Navigating This Book-Part III, “People”
  - benefits of, Preface, Advantages of Microservices-Composability, And Microservice Architecture
  - best uses for, Should I Use Microservices?-Where They Work Well
  - challenges of, Microservice Pain Points-Data Consistency
  - defined, What Are Microservices?, Glossary
  - defining standards for, The Required Standard-Architectural Safety
  - future directions, Looking Forward
  - goal setting, Have a Goal, Strategic Goals, Moving to Microservices
  - key concepts, Key Concepts of Microservices-Alignment of Architecture and Organization
  - monolithic systems, The Monolith-Advantages of Monoliths
  - overview of, Microservices at a Glance-Microservices at a Glance
  - role of technology in, Enabling Technology-Public Cloud and Serverless
  - versus service-oriented architecture, Microservices at a Glance
- middleware, Middleware
- mob programming, Ensemble programming
- mocking downstream collaborators, Mocking or Stubbing
- modeling microservices (see also development; microservices)
  - alternatives to business domain boundaries, Alternatives to Business Domain Boundaries-Organizational
  - domain-driven design, Just Enough Domain-Driven Design-The Case for Domain-Driven Design for Microservices
  - microservice boundaries, What Makes a Good Microservice Boundary?-The Interplay of Coupling and Cohesion
  - mixing models and exceptions, Mixing Models and Exceptions
  - types of coupling, Types of Coupling-Content Coupling
- models
  - branching during development, Branching Models
  - hidden, Hidden models
  - shared, Shared models
- modular microservices, Pluggable, Modular Microservices-Ensemble programming
- modular monoliths, The Modular Monolith
- monitoring (see also observability)
  - automated anomaly detection, The Expert in the Machine
  - challenges of microservices, Monitoring and Troubleshooting, Disruption, Panic, and Confusion
  - defining standards for, Monitoring

  - getting started, Getting Started
  - multiple servers, multiple servers design, Multiple Services, Multiple Servers
  - versus observability, Observability Versus Monitoring-The Pillars of Observability? Not So Fast, Monitoring and Observability
  - real user monitoring, Real user monitoring
  - semantic monitoring, Semantic Monitoring-Real user monitoring
  - single microservice, single server design, Single Microservice, Single Server
  - single service, multiple servers design, Single Microservice, Multiple Servers
  - standardization, Standardization
  - tool selection, Selecting Tools-Suitable for Your Scale
- monolithic frontend pattern, Pattern: Monolithic Frontend-When to Use It
- monolithic systems (see also decomposition)
  - advantages of, Advantages of Monoliths
  - coexisting with microservices, The Monolith Is Rarely the Enemy
  - defined, The Monolith
  - delivery contention and, Monoliths and Delivery Contention
  - distributed, The Distributed Monolith
  - versus legacy architecture, Advantages of Monoliths
  - modular, The Modular Monolith
  - single-process, The Single-Process Monolith
- monorepo approach, Pattern: Monorepo-Where to use this pattern, Glossary
- mountebank stub/mock server, A Smarter Stub Service
- MTBF (mean time between failures), Mean Time to Repair over Mean Time Between Failures?
- MTTR (mean time to repair), Mean Time to Repair over Mean Time Between Failures?
- multirepo approach, Pattern: One Repository per Microservice (aka Multirepo)-Where to use this pattern, Glossary
- multitenancy (Kubernetes), Multitenancy and Federation-Multitenancy and Federation
- mutual authentication, Client identity
- mutual TLS, Client identity

N

- National Institute of Standards and Technology (NIST), The Five Functions of Cybersecurity
- nonfunctional requirements, Cross-Functional Testing
- notification-based invalidation, Notification-based

O

- observability (see also monitoring)
  - alerting, Alerting-Toward better alerting
  - building blocks for, Building Blocks for Observability
  - distributed tracing, Distributed Tracing-Implementing distributing tracing
  - log aggregation, Log Aggregation-Shortcomings
  - metrics aggregation, Metrics Aggregation-Implementations
  - versus monitoring, Observability Versus Monitoring-The Pillars of Observability? Not So Fast, Monitoring and Observability
  - semantic monitoring, Semantic Monitoring-Real user monitoring
  - system health, Are We Doing OK?-Error budgets
  - testing in production, Testing in Production-Chaos engineering
- on-demand provisioning systems, Scaling
- open source development, Branching Models, Internal Open Source
- OpenAPI, Challenges, Explicit Interface, Explicit Schemas
- openapi-diff, Catch Accidental Breaking Changes Early
- OpenID Connect, Common Single Sign-On Implementations
- Operator, Helm, Operators, and CRDs, Oh My!
- orchestrated sagas, Implementing Sagas-Orchestrated sagas, Mixing styles-Should I use choreography or orchestration (or a mix)?, Workflow
- orchestration, Glossary
- organizational alignment, Organizational Alignment, Organizational-Organizational
- organizational structures
  - benefits of autonomy, On Autonomy
  - benefits of loosely coupled organizations, Understanding Conway's Law
  - case study, Case Study: realestate.com.au-Case Study: realestate.com.au
  - Conway's law, Conway's Law-Netflix and Amazon
  - enabling teams, Enabling Teams-The paved road
  - geographical distribution, Geographical Distribution
  - human dynamics and, People
  - impact of system design on organizations, Conway's Law in Reverse
  - internal open source, Internal Open Source-Tooling
  - loosely coupled organizations, Loosely Coupled Organizations
  - orphaned services, The Orphaned Service
  - overview of, Organization
  - pluggable, modular microservices, Pluggable, Modular Microservices-Ensemble programming
  - shared microservices, Shared Microservices-Delivery Bottlenecks

    - small teams, large organization, Small Teams, Large Organization-Small Teams, Large Organization
    - strong versus collective ownership, Strong Versus Collective Ownership-Balancing Models
    - team size, Team Size
- orphaned services, The Orphaned Service
- ownership models
    - changing ownership, Changing ownership
    - overview of, Ownership Models-Drivers for Dedicated Frontend Teams
    - pair programming, Change Reviews-Synchronous versus asynchronous code reviews
    - strong versus collective ownership, Glossary, Glossary

P

- Pact testing tool, Pact
- page-based decomposition, Pattern: Page-Based Decomposition-Where to Use It
- pair programming, Change Reviews-Synchronous versus asynchronous code reviews
- parallel run pattern, Parallel Run, Parallel Run, Parallel run
- parallel, versus sequential calls, Implementation: Synchronous Versus Asynchronous
- partition tolerance, Sacrificing Partition Tolerance?
- partitions, Implementation
- pass-through coupling, Pass-Through Coupling-Pass-Through Coupling
- passwords, salted password hashing, Go with the well known
- patching, Patching-Patching
- pathological coupling, Content Coupling
- paved road concept, The paved road, Governance and the Paved Road-The Paved Road at Scale
- performance tests, Performance Tests
- personally identifiable information (PII), Glossary
- Platform as a Service (PaaS), Platform as a Service (PaaS)
- platform teams, The platform team
- pods (Kubernetes), A Simplified View of Kubernetes Concepts
- preproduction validation, Types of Tests
- preventative control, Defense in Depth, Glossary
- principal, Authentication and Authorization, Glossary
- principle of defense in depth, Defense in Depth
- principle of least privilege, Principle of Least Privilege

- private keys, scanning for, Revocation
- production experiments, Production Experiments
- production monitoring tools, Implementations
  - (see also monitoring)
- programming approaches
  - ensemble programming, Ensemble programming
  - mob programming, Ensemble programming
  - structured programming, Types of Coupling
- progressive delivery, Progressive Delivery-Parallel Run
- Prometheus, Implementations
- Protolock, Catch Accidental Breaking Changes Early
- Puppet, Which Deployment Option Is Right for You?

Q

- questions and comments, How to Contact Us
- queue-based systems, Topics and queues

R

- real user monitoring, Real user monitoring
- real-time monitoring, Real-Time
- rebound, ability to, Rebound
- redundancy, Redundancy
- remote procedure calls (RPCs), Remote Procedure Calls-Where to use it
- replica sets (Kubernetes), A Simplified View of Kubernetes Concepts
- reporting databases, Reporting Database
- reporting, challenges of microservices, Reporting
- Representational State Transfer (REST), REST-Where to use it
- request caches, Request cache
- request-response communication, Pattern: Request-Response Communication-Where to Use It, Communication Styles
- requests
  - versus commands, Pattern: Request-Response Communication
  - defined, Glossary
- resiliency
  - blameless post-mortems, Blame
  - CAP theorem, CAP Theorem-And the Real World
  - challenges of, Failure Is Everywhere
  - chaos engineering, Chaos Engineering-From Robustness to Beyond

  - core concepts of, What Is Resiliency?-And Microservice Architecture
  - cross-functional requirements and, How Much Is Too Much?
  - impact on functionality, Degrading Functionality
  - overview of, Resiliency
  - risk management, Spreading Your Risk
  - stability patterns, Stability Patterns-Idempotency
- response time, How Much Is Too Much?
- responses, Glossary
- responsibilities, understanding, Architecture in a Stream-Aligned Organization
- responsive control, Defense in Depth, Glossary
- REST-over-HTTP-based APIs, Where to use it
- retries, Retries
- risk management, Spreading Your Risk
- robustness
  - assuring, Robustness
  - benefits of caching, For Robustness
  - challenges of improving, Robustness
  - chaos engineering and, Chaos Engineering
  - defined, Glossary
  - determining which options to use, Summary
  - principle of, Tolerant Reader
- robustness tests, Robustness Tests
- rollbacks
  - reducing, Reordering workflow steps to reduce rollbacks
  - semantic rollbacks, Saga rollbacks
  - when using sagas, Saga rollbacks-Mixing fail-backward and fail-forward situations
- root cause analysis, Blame
- RPCs (remote procedure calls), Remote Procedure Calls-Where to use it

S

- sagas
  - benefits of, Sagas
  - core concept of, Sagas
  - defined, Glossary
  - versus distributed transactions, Sagas Versus Distributed Transactions
  - failure modes, Saga Failure Modes-Mixing fail-backward and fail-forward situations

  - implementing, Implementing Sagas-Should I use choreography or orchestration (or a mix)?
  - lack of atomicity in, Sagas
  - limitations of, Saga Failure Modes
  - role in microservices, Sagas, Workflow
- Salt, Which Deployment Option Is Right for You?
- salted password hashing, Go with the well known
- scaling
  - autoscaling, Autoscaling
  - avoiding premature optimization, Start Small
  - benefits of, Scaling
  - caching, Caching-Cache Poisoning: A Cautionary Tale
  - CQRS and event sourcing, Start Small
  - data partitioning, Data Partitioning-Limitations
  - databases during deployment, Database deployment and scaling
  - four axes of, The Four Axes of Scaling
  - functional decomposition, Functional Decomposition-Limitations
  - horizontal duplication, Horizontal Duplication-Limitations
  - microservice templates and, The Paved Road at Scale
  - monitoring and, Suitable for Your Scale
  - overview of, Scaling
  - system redesigns and, Starting Again
  - using combined models for, Combining Models
  - vertical scaling, Vertical Scaling-Limitations
- schemas, Make Your Interface Explicit, Schemas-Should You Use Schemas?
- Schrödinger backup, Backups
- SCSs (self-contained systems), When to Use It
- secrets
  - aspects requiring management, Secrets
  - examples of, Secrets
  - tooling to manage, Secrets
- security
  - areas of concern, Security
  - authentication and authorization, Authentication and Authorization-Challenges
  - challenges of microservices, Security, Security
  - core principles, Core Principles-Build Security into the Delivery Process
  - five functions of cybersecurity, The Five Functions of Cybersecurity-Recover
  - foundations of application security, Foundations of Application Security-Rebuild
  - implicit versus zero trust, Implicit Trust Versus Zero Trust-It's a Spectrum

  - overview of, Security
  - securing data, Securing Data-Encrypt backups
  - types of security controls, Defense in Depth
- self-contained systems (SCSs), When to Use It
- self-describing systems, The Self-Describing System
- semantic breakages, Structural Versus Semantic Contract Breakages
- semantic monitoring, Semantic Monitoring-Real user monitoring
- semantic rollbacks, Saga rollbacks
- semantic versioning, Explicit Interface
- sequential, versus parallel calls, Implementation: Synchronous Versus Asynchronous
- serialization
  - binary formats, Binary Formats
  - textual formats, Textual Formats
- server identity, Server identity
- server-side caching, Server-side-Server-side
- serverless
  - benefits of, Public Cloud and Serverless
  - defined, Platform as a Service (PaaS), Glossary
  - Function as a Service (FaaS), Function as a Service (FaaS)-The way forward
- service discovery, Service Discovery-Don't Forget the Humans!
- service meshes, Service Meshes and API Gateways-Service Meshes and API Gateways, Service Meshes-What About Other Protocols?, And Knative, Glossary
- service providers, Common Single Sign-On Implementations
- service tests, Service Tests, Implementing Service Tests-A Smarter Stub Service
- service-level agreement (SLA), Service-level agreement, Glossary
- service-level indicator (SLI), Service-level indicators, Glossary
- service-level objective (SLO), Service-level objectives, How Much Is Too Much?, Glossary
- service-oriented architecture (SOA), Microservices at a Glance, Glossary
- service-to-service authentication, Service-to-Service Authentication
- services (Kubernetes), A Simplified View of Kubernetes Concepts
- services, documenting, Documenting Services-The Self-Describing System
- shards, Implementation
- shared models, Shared models, Shared Microservices-Delivery Bottlenecks
- Simple Notification Service (SNS), Choices
- Simple Queue Service (SQS), Choices
- single sign-on (SSO), Common Single Sign-On Implementations-Single Sign-On Gateway
- single-page application (SPA), Glossary

- single-process monoliths, The Single-Process Monolith
- size, Size
- smoke tests, Smoke tests
- Snyk, Build Security into the Delivery Process
- social contract, The Social Contract
- software architecture, What Is Software Architecture?
- Software as a Service (SaaS), Where They Work Well
- software engineering, What's in a Name?
- source code branching, Branching Models
- SPA (single-page application), Glossary
- spans (distributed tracing), How it works
- Spring Cloud Contract, Other options
- SSO (single sign-on), Common Single Sign-On Implementations-Single Sign-On Gateway
- stability patterns
  - bulkheads, Bulkheads
  - case study, Stability Patterns
  - circuit breakers, Circuit Breakers-Circuit Breakers
  - idempotency, Idempotency
  - isolation, Isolation
  - middleware, Middleware
  - redundancy, Redundancy
  - retries, Retries
  - time-outs, Time-Outs-Time-Outs
- strangler fig pattern, Strangler Fig Pattern, Stability Patterns
- strategic goals, Strategic Goals
- stream-aligned teams, Alignment of Architecture and Organization, Toward Stream-Aligned Teams-Working Through Technical Challenges, Loosely Coupled Organizations, Architecture in a Stream-Aligned Organization-Architecture in a Stream-Aligned Organization, Glossary
- streaming, role in microservices, Streaming
- strong ownership, Strong Ownership-How far does strong ownership go?, Glossary
- structural breakages, Structural Versus Semantic Contract Breakages
- structured programming, Types of Coupling
- stubbing downstream collaborators, Mocking or Stubbing
- sustained adaptability, Sustained Adaptability, Chaos Engineering, Summary, Glossary
- synchronous blocking calls, Pattern: Synchronous Blocking-Where to Use It

- synthetic transactions, Synthetic transactions-Implementing synthetic transactions
- system health, Are We Doing OK?-Error budgets
- systems architects (see evolutionary architects)

T

- tailored microservices templates, Tailored Microservice Template
- target audience, Who Should Read This Book
- teams
  - building, Building a Team
  - community of practice (CoP) groups, Communities of Practice
  - enabling teams, Sharing Specialists, Enabling Teams-The paved road, Architecture in a Stream-Aligned Organization
  - ensuring consistency across, Ensuring Consistency
  - full stack teams, Toward Stream-Aligned Teams
  - geographical distribution, Geographical Distribution
  - platform teams, The platform team
  - size of, Team Size
  - small teams, large organization, Small Teams, Large Organization-Small Teams, Large Organization
  - stream-aligned teams, Toward Stream-Aligned Teams-Working Through Technical Challenges, Loosely Coupled Organizations, Architecture in a Stream-Aligned Organization-Architecture in a Stream-Aligned Organization
  - strong versus collective ownership, At a Team Level Versus an Organizational Level
  - team APIs, Small Teams, Large Organization
  - two-pizza teams, Netflix and Amazon
  - understanding responsibilities, Architecture in a Stream-Aligned Organization
- technical debt, Technical Debt
- technology choices
  - GraphQL, GraphQL-Where to use it
  - message brokers, Message Brokers-Kafka
  - options explored, Technology Choices
  - remote procedure calls (RPCs), Remote Procedure Calls-Where to use it
  - Representational State Transfer (REST), REST-Where to use it
  - stream-aligned teams and, Working Through Technical Challenges
- technology coupling, Technology coupling
- technology heterogeneity, Technology Heterogeneity, Technology
- technology overload, Technology Overload

- technology-facing tests, Types of Tests
- templates, Tailored Microservice Template
- temporal coupling, Domain Coupling
- testing
  - A/B tests, A/B testing
  - canary releases, Canary release
  - challenges of microservices, Testing
  - chaos engineering, Chaos engineering
  - consumer-driven contract testing, Catch Accidental Breaking Changes Early
  - consumer-driven contracts (CDCs), Contract Tests and Consumer-Driven Contracts (CDCs)-The Final Word
  - contract tests, Contract Tests and Consumer-Driven Contracts (CDCs)
  - cross-functional testing, Cross-Functional Testing-Robustness Tests
  - developer experience, Developer Experience
  - end-to-end test alternatives, Should You Avoid End-to-End Tests?-The Final Word
  - end-to-end test implementation, Implementing (Those Tricky) End-to-End Tests-Lack of Independent Testability
  - holistic approach to, Summary
  - manual exploratory testing, Types of Tests
  - overview of, Testing
  - Pact testing tool, Pact
  - parallel run pattern, Parallel run
  - performance tests, Performance Tests
  - preproduction to in-production testing, From Preproduction to In-Production Testing-Mean Time to Repair over Mean Time Between Failures?, Testing in Production-Chaos engineering
  - robustness tests, Robustness Tests
  - service test implementation, Implementing Service Tests-A Smarter Stub Service
  - smoke tests, Smoke tests
  - synthetic transactions, Synthetic transactions-Implementing synthetic transactions
  - test scope, Test Scope-Trade-Offs
  - types of tests, Types of Tests-Types of Tests
- threat modeling, Glossary
- three pillars of observability, The Pillars of Observability? Not So Fast
- tight coupling, Coupling
- tightly coupled organizations, Loosely and tightly coupled organizations
- time to live (TTL), Domain Name System (DNS), Time to live (TTL)

- time-outs, Stability Patterns-Time-Outs
- tooling, Tooling
- topic-based systems, Topics and queues
- traces (distributed tracing), How it works
- transactions (see database transactions)
- trunk-based development, Branching Models, Glossary
- trust, Trust
- two-phase commit algorithms (2PCs), Distributed Transactions—Two-Phase Commits-Distributed Transactions—Two-Phase Commits
- two-pizza teams, Netflix and Amazon
- type 2 virtualization, Cost of virtualization

U

- ubiquitous language, Ubiquitous Language, Glossary
- unit tests, Unit Tests
- upstream authorization, Centralized, Upstream Authorization
- usage, tracking, Tracking Usage
- user credentials, User credentials
- user experience, Make Your Service Simple for Consumers
- user interfaces (UIs)
  - adapting to device constraints, Constraints
  - backend for frontend (BFF) pattern, Pattern: Backend for Frontend (BFF)-When to Use
  - central aggregating gateway pattern, Pattern: Central Aggregating Gateway-When to Use It
  - developments over time, User Interfaces
  - GraphQL, GraphQL-GraphQL
  - hybrid approach, A Hybrid Approach
  - melding of digital and mobile UIs, Toward Digital
  - micro frontend pattern, Pattern: Micro Frontends-When to Use It
  - monolithic frontend pattern, Pattern: Monolithic Frontend-When to Use It
  - overview of, User Interfaces
  - ownership models, Ownership Models-Drivers for Dedicated Frontend Teams
  - page-based decomposition, Pattern: Page-Based Decomposition-Where to Use It
  - stream-aligned teams, Toward Stream-Aligned Teams-Working Through Technical Challenges, Loosely Coupled Organizations
  - widget-based decomposition, Pattern: Widget-Based Decomposition-When to Use It

V

- variations, running multiple, Run multiple variations
- Vault, Secrets
- versioning, Handling Change Between Microservices, Explicit Interface, Coexist Incompatible Microservice Versions, Infrastructure as Code (IAC)
- vertical scaling, Vertical Scaling-Limitations, Glossary
- VFS for Git, Tooling
- virtual machines (VMs), Virtual Machines-Good for microservices?, Glossary
- volatility-based decomposition, Volatility
- voting phase, Distributed Transactions—Two-Phase Commits

W

- Wasm (WebAssembly), Limitations
- Web Component Standard, Communication between in-page widgets
- WebAssembly System Interface (WASI), Limitations
- widget-based decomposition, Pattern: Widget-Based Decomposition-When to Use It
- widgets, Glossary
- Windows containers, Windows containers
- workflow
  - database transactions, Database Transactions-Still ACID, but Lacking Atomicity?
  - distributed transactions, avoiding, Distributed Transactions—Just Say No
  - distributed transactions, two-phase commits, Distributed Transactions—Two-Phase Commits-Distributed Transactions—Two-Phase Commits
  - overview of, Workflow
  - sagas, Sagas-Sagas Versus Distributed Transactions
- write-behind caches, Write-behind
- write-through caches, Write-through

Z

- Zed Attack Proxy (ZAP), Build Security into the Delivery Process
- zero trust, Zero Trust
- zero-downtime deployment, Zero-Downtime Deployment
- ZooKeeper, ZooKeeper

Previous chapter

< Glossary

Next chapter

About the Author >